

PERFORMANCE OPTIMIZATION

This means making the **system work faster and more efficiently**, so results come quickly and resources are not wasted. And also It involves tuning queries, warehouses, storage's , and **workload patterns to reduce execution time and resource usage in snowflake.**

The following topics help guide efforts to improve the performance of Snowflake:

- Exploring execution times
- Optimizing query performance
- Optimizing warehouses for performance
- Optimizing storage for performance
- Analyzing query workloads with Performance Explorer
- Snowflake Optima

Exploring execution times:

This area focuses on understanding how long queries take to run and why. It includes Which tasks are **fast**, Which tasks are **slow**, Where time is being spent.

- Reviewing query execution history
- Analyzing query profiles and execution plans
- Identifying bottlenecks such as slow scans, joins.

Optimizing query performance:

It covers techniques to make SQL queries run faster and use fewer resources. Also It **asking questions in a smarter way** so the system can answer faster. Common practices include:

- Writing efficient SQL (avoiding unnecessary columns, filters, or joins)
- Using proper join types and join order
- Leveraging clustering keys and pruning
- Avoiding expensive operations like large **SELECT *** queries
(The goal is to reduce query execution time and warehouse load)

Optimizing warehouses for performance:

- Optimizing a Snowflake virtual warehouse ensures fast query execution with minimal cost by choosing the right size, scaling strategy, and usage settings. Performance optimization means running queries faster while using credits efficiently.

In this there are some strategies for optimizing warehouses for performance:

- Reduce queues
- Resolve memory spillage
- Increase warehouse size
- Try query acceleration
- Optimize the warehouse cache
- Limit concurrently running queries

Reduce queues:

- Happens when many queries run on a small warehouse.
- Queries wait in a queue(waiting list) → slow performance.

This is the solution:

- Increase warehouse size
- Use **Multi-Cluster Warehouses**
- Separate workloads (ETL vs Reporting)
Result: Faster query start time

For example:

10 users run reports at the same time on a **Small warehouse**. Problem - Queries wait in queue , Slow response. In this manner the queries is start immediately, no waiting.

Solution : `alter warehouse report_wh set warehouse size = 'medium';`

Resolve memory spillage:

- Occurs when queries need more memory than available.
- Data spills to local or remote disk → slower execution.

This is the solution:

- Increase warehouse size
- Optimize SQL (filters, joins, aggregations)

- Avoid large unnecessary result sets
Result: Faster query execution

For example:

A large **JOIN** query spills data to disk . No disk spill .After optimization, the query runs **fully in memory**, resulting in **faster execution**.

Solution : **alter warehouse report_wh set warehouse size = 'large';**

Increase warehouse size:

- When we increase the warehouse size : More CPU,More memory are happens.Through Heavy ETL jobs , Large joins , Aggregations , High user concurrency.

For example:

Daily ETL job takes **45 minutes** on Small warehouse

Solution : **alter warehouse etl_wh set warehouse size = 'xlarge';** It will gain ETL completes in **15 minutes**.

Try query acceleration :

- Query acceleration - Snowflake automatically adds **extra compute resources** for **long-running queries**, without resizing the warehouse.
- **Best for** Complex analytical queries , Large table scans , BI reporting.

For example:

A dashboard query scans **billions of rows**. (Query runs for several minutes)

Solution : snowflake adds compute dynamically → **faster results**.

ALTER WAREHOUSE report_wh SET QUERY_ACCELERATION_MAX_SCALE_FACTOR = 8;

Optimize the warehouse cache:

- Warehouse cache stores recently accessed data in memory and local SSD of the virtual warehouse. **so that repeated queries can run faster without re-reading data from cloud storage.**

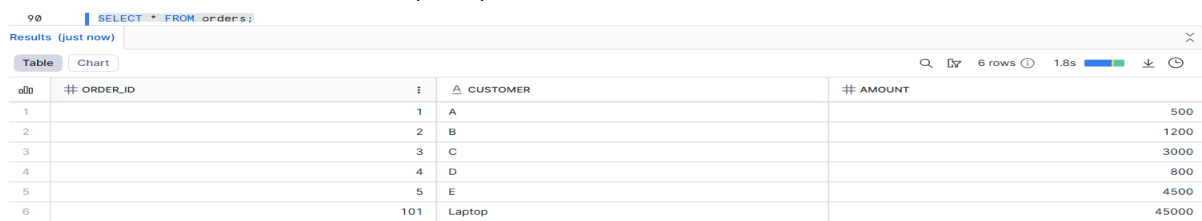
Types of Cache :

Result Cache - Stores query results and also no compute is used for repeated queries

- When its work : Same query ,No data change ,same role & warehouse context.

Eg : `Select * from orders;`

- First run → scans table (slow)



90 | `SELECT * FROM orders;`

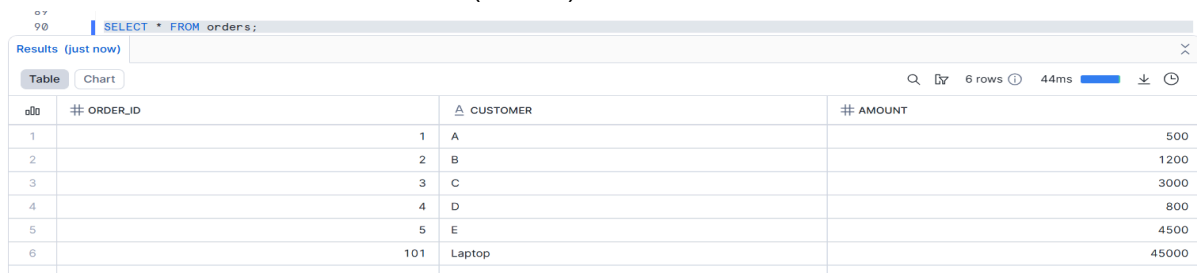
Results (just now)

Table | Chart

6 rows 1.8s

#	ORDER_ID	CUSTOMER	AMOUNT
1	1	A	500
2	2	B	1200
3	3	C	3000
4	4	D	800
5	5	E	4500
6	101	Laptop	45000

- Second run → **result cache** (instant)



97 | `SELECT * FROM orders;`

Results (just now)

Table | Chart

6 rows 44ms

#	ORDER_ID	CUSTOMER	AMOUNT
1	1	A	500
2	2	B	1200
3	3	C	3000
4	4	D	800
5	5	E	4500
6	101	Laptop	45000

Final output : Millisecond response

Local Disk Cache - Stores micro-partitions on warehouse SSD and it avoids reading from cloud storage to local disk cache.

- When its works : Same warehouse ,Warehouse is not suspended and Similar queries on same data

Eg : A dashboard repeatedly queries last 7 days of sales. `Select sum(amount) From sales Where order_date >= current date - 7;`

- First run → data loaded from cloud storage

```

199 SELECT SUM(amount) AS total_sales_last_7_days
200 FROM sales
201 WHERE order_date >= CURRENT_DATE - 7;
202

```

Results (just now)

Table Chart

1 row 573ms

TOTAL_SALES_LAST_7_DAYS
5801.25

- Next runs → **local disk cache**

```

198 SELECT SUM(amount) AS total_sales_last_7_days
199 FROM sales
200 WHERE order_date >= CURRENT_DATE - 7;
201
202

```

Results (just now)

Table Chart

1 row 47ms

TOTAL_SALES_LAST_7_DAYS
5801.25

Final output : Faster execution

Remote Disk Cache :

- It stores **data on Snowflake-managed cloud storage**, instead of the local SSD of a warehouse.
- It's **slower than local warehouse cache** but **faster than reading directly from raw table storage** (S3, Azure Blob, or GCS).
- Used **when the local cache is unavailable**, e.g., after a warehouse restart or when a new cluster starts.
- **When its works** : Warehouse restarted and Local cache lost

Cache hierarchy in Snowflake :

Result Cache → Local Disk Cache (wh - SSD) → Remote Disk Cache → Cloud Storage.

Practical example :

Warehouse Status	Query Execution
Warehouse running, data in local SSD	0.2 sec
Warehouse restarted, data in remote cache	1–2 sec
Warehouse restarted, data not in remote cache	10–12 sec (full scan)

Limit concurrently running queries:

Limiting concurrently running queries means controlling **how many queries can run at the same time** on a Snowflake warehouse.

- If too many queries run at once, some will wait in a queue.
- Limiting concurrency **prevents the warehouse from getting overloaded** and keeps performance smooth.

For Example:

We have a **small warehouse**.

A SMALL warehouse can run **only a limited number of queries at the same time**. Extra queries will **wait in a queue**.

Step 1: Create a SMALL Warehouse

```
205
206 CREATE OR REPLACE WAREHOUSE small_wh
207 WITH WAREHOUSE_SIZE = 'SMALL'
208 AUTO_SUSPEND = 60
209 AUTO_RESUME = TRUE;
```

Results (just now)

Table Chart 1 row 172ms

#	status
1	Warehouse SMALL_WH successfully created.

This warehouse can run **only a few queries concurrently**.

Step 2: Run Multiple Queries at the Same Time - Assume **3 users** run queries together:

```
211 -- User 1
212 SELECT COUNT(*) FROM sales;
213
214 -- User 2
215 SELECT SUM(amount) FROM sales;
216
217 -- User 3
218 SELECT * FROM sales LIMIT 10;
```

Results (just now)

Table Chart 5 rows 28ms

#	ORDER_ID	AMOUNT	ORDER_DATE
1	1	1500.00	2026-01-04
2	2	2200.50	2026-01-02
3	3	900.00	2025-12-31

Step 3: What Happens:

- **Query 1 & Query 2** → run immediately
- **Query 3** → **WAITs in QUEUE**

This waiting happens because **concurrent queries are limited**.

Step 4: Reduce Waiting by Allowing More Concurrency

221	ALTER WAREHOUSE small_wh SET WAREHOUSE_SIZE = 'MEDIUM';
222	
223	ALTER WAREHOUSE small_wh
224	SET
225	MIN_CLUSTER_COUNT = 1,
226	MAX_CLUSTER_COUNT = 3;
227	

Results (just now)

TableChart

1 row74ms

status

1Statement executed successfully.

- Snowflake automatically adds clusters when many queries run.
- More queries run **at the same time**.
- **Less queueing**.

Limiting concurrently running queries means only a fixed number of queries can run at the same time on a warehouse extra queries wait in a queue to avoid overload.

Optimizing storage for performance:

- It means storing data in a way that **Snowflake can read less data and run queries faster**. Snowflake already optimizes a lot automatically, but **how you design and manage data matters**.

Key Techniques to Optimize Storage for Performance:

- Micro-Partition Pruning (Automatic)
- Cluster Data for Large Tables (Manual Optimization)
- Avoid Frequent DELETES (Use TRUNCATE or CTAS)
- Use Appropriate Data Types
- Use Result Cache (Free Performance Boost)

Micro-Partition Pruning (Automatic)

- Snowflake stores data in micro-partitions
- Each micro-partition has metadata (min/max values)

Eg : Snowflake scans **only micro-partitions** containing that date and also Does NOT scan the entire sales table.

Cluster Data for Large Tables (Manual Optimization)

- Tables are **very large**
- Queries filter on same columns repeatedly.

Eg : Related data stored together ,Faster scans for filtered queries and also Use clustering only for **large & frequently queried tables**.

Avoid Frequent DELETES (Use TRUNCATE or CTAS):

In **Snowflake**, tables are stored as **immutable micro-partitions**. When a DELETE statement is executed, Snowflake **does not physically remove data immediately**. Instead, it marks rows as deleted (called **tombstones**). This causes:

- More micro-partitions to be scanned
- Slower query performance
- Higher compute and storage cost

Therefore, frequent DELETE operations should be avoided.

Use TRUNCATE (When All Rows Must Be Removed)

TRUNCATE removes **all rows instantly** without creating tombstones .(**Truncate table sales;**)

Why TRUNCATE is better :

- Very fast operation
- No tombstones
- Better storage and query performance
- Table structure remains intact

Use CTAS (When Partial Data Must Be Removed)

- For deleting **specific rows** from large tables, rebuild the table using **CTAS (Create Table As Select)**.

```
CREATE OR REPLACE TABLE sales AS SELECT * FROM sales  
WHERE order_date >= '2024-01-01';
```

Why CTAS is better

- Creates clean micro-partitions
- No deleted data left behind
- Faster queries
- Recommended for large tables

Use Appropriate Data Types :

- Using appropriate data types means storing data in the correct format and size so Snowflake uses less space and can process queries faster.

Simple Hierarchy :

Wrong Data Type	Problem	Correct Data Type
VARCHAR for numbers	More space, slow	NUMBER
VARCHAR for dates	No pruning	DATE
Large VARCHAR(1000)	Wastes space	VARCHAR(50)

Use Result Cache (Free Performance Boost):

- Snowflake stores the result of a previously executed query and returns it instantly when the same query is run again .This improves performance with **zero compute cost** as long as the data has not changed and result Cache returns previously query.
- Snowflake automatically reuses cached query results to deliver answers instantly for repeated queries .Speeds up performance and saves compute credits.

Analyzing query workloads with Performance Explorer :

- **Performance Explorer** is a Snowflake UI feature used to **analyze how queries run on a warehouse over time** and how efficiently compute resources are used.

In this there Few Key Information we Can Analyze:

- Query Load Over Time
- Long-Running Queries
- Concurrency & Queuing
- Warehouse Utilization

Query Load Over Time:

- Shows when queries start and finish
- Helps identify peak usage hours
- Useful for capacity planning

Long-Running Queries:

- Identifies queries that take more time
- Helps find poorly written or unoptimized SQL.

Concurrency & Queuing:

- Shows how many queries run **at the same time**
- Helps detect **queue delays**
- Useful to decide:
 - Increase warehouse size
 - Use multi-cluster warehouses

Warehouse Utilization

- Displays how efficiently a warehouse is used
- Helps avoid:
 - Over-sized warehouses (waste)
 - Under-sized warehouses (slow queries)

Snowflake Optima:

Snowflake Optimization means **making Snowflake run faster and cheaper** by using the right features and best practices. **Snowflake Optimization Focuses On these kind:**

- Storage Optimization
- Query Optimization
- Warehouse Optimization
- Cost Optimization

Storage Optimization:

- Store data efficiently
- Use correct data types
- Benefit from micro-partition pruning

Result: Less data scanned, faster queries.

Query Optimization:

- Write efficient SQL
- Avoid SELECT *
- Use filters and Result Cache

Result: Faster execution, less compute usage.

Warehouse Optimization:

- Choose correct warehouse size
- Handle concurrency properly
- Separate ETL and reporting

Result: No query delays, better performance.

Cost Optimization:

- Auto-suspend unused warehouses
- Monitor usage with Performance Explorer

Result: Reduced Snowflake credits.